

h e p i a

Haute école du paysage, d'ingénierie
et d'architecture de Genève

—

OS pour le déploiement de services conteneurisés

Défense du projet de bachelor

Frédéric ARROYO

2026-09-01

HEPIA // HES-SO Genève

h e p i a

Haute école du paysage, d'ingénierie
et d'architecture de Genève

- Déploiements modestes

Introduction — Problématique

- Début: 00:20 Fin: **01:50**
 - **Ce projet s'intéresse l'administration d'environnement modeste**
 - Par modestes, il faut entendre des environnements administrés par une seule personne, sans clustering, sans déploiement multirégion, et sans redondance particulière.
 - C'est typiquement le genre de déploiement que vous pourriez avoir si vous hébergez un site à titre personnel ou un petit projet avec une dizaine, une centaine, voire un millier d'utilisateurs tout au plus.
- >>>

- Déploiements modestes
- Basé sur les conteneurs

Introduction — Problématique

- Début: 00:20 Fin: **01:50**
- Modestes, oui, mais aussi modernes: les déploiements visés se basent sur la conteneurisation pour gérer l'application.
- Cette conteneurisation permet de s'affranchir d'une grande partie de la complexité dans l'administration de systèmes, à savoir l'installation et la gestion des applications elles-mêmes. >>>

- Déploiements modestes
- Basé sur les conteneurs

- Début: 00:20 Fin: **01:50**
- Toutefois, il reste nécessaire de configurer et de maintenir l'hôte sur lequel les conteneurs s'exécutent. >>>
- Cette configuration, dans ce genre de déploiement, suit un ensemble assez commun d'étapes, à commencer par l'installation du système d'exploitation. >>>
- Il est ensuite nécessaire de le configurer (accès SSH, configuration réseau, installation des paquets) >>>
- puis d'y installer l'environnement d'exécution de conteneurs, par exemple Docker. >>>
- Une fois tout cela fait, vous pourrez y déployer vos conteneurs. >>>

- Déploiements modestes
- Basé sur les conteneurs
- Étapes communes

Introduction — Problématique

- Début: 00:20 Fin: **01:50**
- Toutefois, il reste nécessaire de configurer et de maintenir l'hôte sur lequel les conteneurs s'exécutent. >>>
- Cette configuration, dans ce genre de déploiement, suit un ensemble assez commun d'étapes, à commencer par l'installation du système d'exploitation. >>>
- Il est ensuite nécessaire de le configurer (accès SSH, configuration réseau, installation des paquets) >>>
- puis d'y installer l'environnement d'exécution de conteneurs, par exemple Docker. >>>
- Une fois tout cela fait, vous pourrez y déployer vos conteneurs. >>>

- Déploiements modestes
- Basé sur les conteneurs
- Étapes communes
 1. Installer l'OS

Introduction — Problématique

- Début: 00:20 Fin: **01:50**
- Toutefois, il reste nécessaire de configurer et de maintenir l'hôte sur lequel les conteneurs s'exécutent. >>>
- Cette configuration, dans ce genre de déploiement, suit un ensemble assez commun d'étapes, à commencer par l'installation du système d'exploitation. >>>
- Il est ensuite nécessaire de le configurer (accès SSH, configuration réseau, installation des paquets) >>>
- puis d'y installer l'environnement d'exécution de conteneurs, par exemple Docker. >>>
- Une fois tout cela fait, vous pourrez y déployer vos conteneurs. >>>

- Déploiements modestes
- Basé sur les conteneurs
- Étapes communes
 1. Installer l'OS
 2. Configurer le système (SSH, réseau, paquets)

- Début: 00:20 Fin: **01:50**
- Toutefois, il reste nécessaire de configurer et de maintenir l'hôte sur lequel les conteneurs s'exécutent. >>>
- Cette configuration, dans ce genre de déploiement, suit un ensemble assez commun d'étapes, à commencer par l'installation du système d'exploitation. >>>
- Il est ensuite nécessaire de le configurer (accès SSH, configuration réseau, installation des paquets) >>>
- puis d'y installer l'environnement d'exécution de conteneurs, par exemple Docker. >>>
- Une fois tout cela fait, vous pourrez y déployer vos conteneurs. >>>

- Déploiements modestes
- Basé sur les conteneurs
- Étapes communes
 1. Installer l'OS
 2. Configurer le système (SSH, réseau, paquets)
 3. Installer le runtime de conteneur

- Début: 00:20 Fin: **01:50**
- Toutefois, il reste nécessaire de configurer et de maintenir l'hôte sur lequel les conteneurs s'exécutent. >>>
- Cette configuration, dans ce genre de déploiement, suit un ensemble assez commun d'étapes, à commencer par l'installation du système d'exploitation. >>>
- Il est ensuite nécessaire de le configurer (accès SSH, configuration réseau, installation des paquets) >>>
- puis d'y installer l'environnement d'exécution de conteneurs, par exemple Docker. >>>
- Une fois tout cela fait, vous pourrez y déployer vos conteneurs. >>>

- Déploiements modestes
- Basé sur les conteneurs
- Étapes communes
 1. Installer l'OS
 2. Configurer le système (SSH, réseau, paquets)
 3. Installer le runtime de conteneur
 4. Déployer les conteneurs

Introduction — Problématique

- Début: 00:20 Fin: **01:50**
- Toutefois, il reste nécessaire de configurer et de maintenir l'hôte sur lequel les conteneurs s'exécutent. >>>
- Cette configuration, dans ce genre de déploiement, suit un ensemble assez commun d'étapes, à commencer par l'installation du système d'exploitation. >>>
- Il est ensuite nécessaire de le configurer (accès SSH, configuration réseau, installation des paquets) >>>
- puis d'y installer l'environnement d'exécution de conteneurs, par exemple Docker. >>>
- Une fois tout cela fait, vous pourrez y déployer vos conteneurs. >>>

- Déploiements modestes
- Basé sur les conteneurs
- Étapes communes
 1. Installer l'OS
 2. Configurer le système (SSH, réseau, paquets)
 3. Installer le runtime de conteneur
 4. Déployer les conteneurs

Constat

Malgré ces besoins simples, aucun OS ne remplit ce rôle simplement.

Introduction — Problématique

- Début: 00:20 Fin: **01:50**
- L'administration de l'hôte est simple, presque standardisée, et sa finalité est claire. Alors pourquoi aucun système d'exploitation ne se limite à faire exactement ça, et rien de plus? **|||**

- Projet de semestre, octobre $\Rightarrow$ mars)

- Début: 01:50 Fin: **02:50**
- Et je me penche sur cette question depuis mi-octobre
- En effet, avant ce travail, un travail préalable a eu lieu: le projet de semestre, entre octobre et mars. >>>
- Ce travail avait pour but premier d'identifier des solutions potentielles correspondant aux besoins exprimés; il en ressort qu'aucune n'est adéquate. >>>
- Il s'agissait également d'établir les bases de très haut niveau pour l'architecture et le fonctionnement >>>
- ainsi que de me renseigner et de choisir les technologies à employer. >>>

- Projet de semestre, octobre $\Rightarrow$ mars)
 - Identification de solution existante adéquate: **aucune**

- Début: 01:50 Fin: **02:50**
- Et je me penche sur cette question depuis mi-octobre
- En effet, avant ce travail, un travail préalable a eu lieu: le projet de semestre, entre octobre et mars. >>>
- Ce travail avait pour but premier d'identifier des solutions potentielles correspondant aux besoins exprimés; il en ressort qu'aucune n'est adéquate. >>>
- Il s'agissait également d'établir les bases de très haut niveau pour l'architecture et le fonctionnement >>>
- ainsi que de me renseigner et de choisir les technologies à employer. >>>

- Projet de semestre, octobre $\Rightarrow$ mars)
 - Identification de solution existante adéquate: **aucune**
 - Conceptualisation de très haut niveau

- Début: 01:50 Fin: **02:50**
- Et je me penche sur cette question depuis mi-octobre
- En effet, avant ce travail, un travail préalable a eu lieu: le projet de semestre, entre octobre et mars. >>>
- Ce travail avait pour but premier d'identifier des solutions potentielles correspondant aux besoins exprimés; il en ressort qu'aucune n'est adéquate. >>>
- Il s'agissait également d'établir les bases de très haut niveau pour l'architecture et le fonctionnement >>>
- ainsi que de me renseigner et de choisir les technologies à employer. >>>

- Projet de semestre, octobre $\Rightarrow$ mars)
 - Identification de solution existante adéquate: **aucune**
 - Conceptualisation de très haut niveau
 - Recherche des « briques » logicielles

- Début: 01:50 Fin: **02:50**
- Et je me penche sur cette question depuis mi-octobre
- En effet, avant ce travail, un travail préalable a eu lieu: le projet de semestre, entre octobre et mars. >>>
- Ce travail avait pour but premier d'identifier des solutions potentielles correspondant aux besoins exprimés; il en ressort qu'aucune n'est adéquate. >>>
- Il s'agissait également d'établir les bases de très haut niveau pour l'architecture et le fonctionnement >>>
- ainsi que de me renseigner et de choisir les technologies à employer. >>>

- Projet de semestre, octobre $\Rightarrow$ mars)
 - Identification de solution existante adéquate: **aucune**
 - Conceptualisation de très haut niveau
 - Recherche des « briques » logicielles
- IA: tentative pour débogage + amélioration CI/CD

- Début: 01:50 Fin: **02:50**
- Par ailleurs, tant dans le projet de semestre que dans ce projet, l'IA n'a été que très peu utilisée, principalement dans le but de déboguer et d'améliorer certains aspects annexes de l'implémentation. >>>
- Ces tentatives, cela dit, n'ont pas été couronnées de succès. >>>
- Quoi qu'il en soit, aucun code n'a été produit par l'IA, et aucune décision architecturale n'a été prise par elle. >>>

- Projet de semestre, octobre $\Rightarrow$ mars)
 - Identification de solution existante adéquate: **aucune**
 - Conceptualisation de très haut niveau
 - Recherche des « briques » logicielles
- IA: tentative pour débogage + amélioration CI/CD
 - Sans succès

- Début: 01:50 Fin: **02:50**
- Par ailleurs, tant dans le projet de semestre que dans ce projet, l'IA n'a été que très peu utilisée, principalement dans le but de déboguer et d'améliorer certains aspects annexes de l'implémentation. >>>
- Ces tentatives, cela dit, n'ont pas été couronnées de succès. >>>
- Quoi qu'il en soit, aucun code n'a été produit par l'IA, et aucune décision architecturale n'a été prise par elle. >>>

- Projet de semestre, octobre $\Rightarrow$ mars)
 - Identification de solution existante adéquate: **aucune**
 - Conceptualisation de très haut niveau
 - Recherche des « briques » logicielles
- IA: tentative pour débogage + amélioration CI/CD
 - Sans succès
 - **Pas de code ou de décisions architecturales**

- Début: 01:50 Fin: **02:50**
- Par ailleurs, tant dans le projet de semestre que dans ce projet, l'IA n'a été que très peu utilisée, principalement dans le but de déboguer et d'améliorer certains aspects annexes de l'implémentation. >>>
- Ces tentatives, cela dit, n'ont pas été couronnées de succès. >>>
- Quoi qu'il en soit, aucun code n'a été produit par l'IA, et aucune décision architecturale n'a été prise par elle. >>>

- Projet de semestre, octobre $\Rightarrow$ mars)
 - Identification de solution existante adéquate: **aucune**
 - Conceptualisation de très haut niveau
 - Recherche des « briques » logicielles
- IA: tentative pour débogage + amélioration CI/CD
 - Sans succès
 - **Pas de code ou de décisions architecturales**
- Projet personnel

- Début: 01:50 Fin: **02:50**
- Ce choix tient au fait qu'il s'agit avant tout d'un projet personnel qui me tient à cœur: je voulais avant tout voir mes compétences à l'œuvre. **III**

- Début: 02:50 Fin: **04:30**
- Comme je l'ai dit, aucune solution existante ne convient. Dans ce travail, je vais donc vous présenter la solution que j'ai conçue pour répondre à ce besoin. >>>
- La solution que je propose est une distribution Linux spécialisée pour la conteneurisation. >>>

- Distribution Linux spécialisée

- Début: 02:50 Fin: **04:30**
- Comme je l'ai dit, aucune solution existante ne convient. Dans ce travail, je vais donc vous présenter la solution que j'ai conçue pour répondre à ce besoin. >>>
- La solution que je propose est une distribution Linux spécialisée pour la conteneurisation. >>>

- Distribution Linux spécialisée
- Surface **minimale**: le strict nécessaire pour les conteneurs

Introduction — Solution

- Début: 02:50 Fin: **04:30**
- C'est-à-dire une solution aussi minimale que possible, dans le but de déployer rapidement et simplement des conteneurs. >>>
- Et par minimale, je vais ici à l'extrême en retirant d'abord toute forme d'accès interactif: pas de SSH, pas de shell, et pas de commandes. >>>
- Pour remplacer tout cela, le système expose une API. Cette API est principalement en lecture seule, à l'exception d'un élément: la gestion de la configuration elle-même. >>>

- Distribution Linux spécialisée
- Surface **minimale**: le strict nécessaire pour les conteneurs
 - Pas d'SSH, de shell, ou de commandes

Introduction

— Solution

- Début: 02:50 Fin: **04:30**
- C'est-à-dire une solution aussi minimale que possible, dans le but de déployer rapidement et simplement des conteneurs. >>>
- Et par minimale, je vais ici à l'extrême en retirant d'abord toute forme d'accès interactif: pas de SSH, pas de shell, et pas de commandes. >>>
- Pour remplacer tout cela, le système expose une API. Cette API est principalement en lecture seule, à l'exception d'un élément: la gestion de la configuration elle-même. >>>

- Distribution Linux spécialisée
- Surface **minimale**: le strict nécessaire pour les conteneurs
 - Pas d'SSH, de shell, ou de commandes
 - Piloté par API

Introduction

— Solution

- Début: 02:50 Fin: **04:30**
- C'est-à-dire une solution aussi minimale que possible, dans le but de déployer rapidement et simplement des conteneurs. >>>
- Et par minimale, je vais ici à l'extrême en retirant d'abord toute forme d'accès interactif: pas de SSH, pas de shell, et pas de commandes. >>>
- Pour remplacer tout cela, le système expose une API. Cette API est principalement en lecture seule, à l'exception d'un élément: la gestion de la configuration elle-même. >>>

- Distribution Linux spécialisée
- Surface **minimale**: le strict nécessaire pour les conteneurs
 - Pas d'SSH, de shell, ou de commandes
 - Piloté par API
- Fichier de configuration unique

- Début: 02:50 Fin: **04:30**
- Cette configuration passe par un fichier unique: tout y est centralisé et défini, que ce soit les conteneurs, le réseau, le stockage, ou l'installation. >>>
- Le but est d'avoir un système entièrement déclaratif: vous déclarez simplement l'état dans lequel vous voulez que votre système soit, et il s'occupe lui-même d'y parvenir, sans que vous ayez à effectuer la moindre action. >>>

- Distribution Linux spécialisée
- Surface **minimale**: le strict nécessaire pour les conteneurs
 - Pas d'SSH, de shell, ou de commandes
 - Piloté par API
- Fichier de configuration unique
 - Système déclaratif

Introduction

— Solution

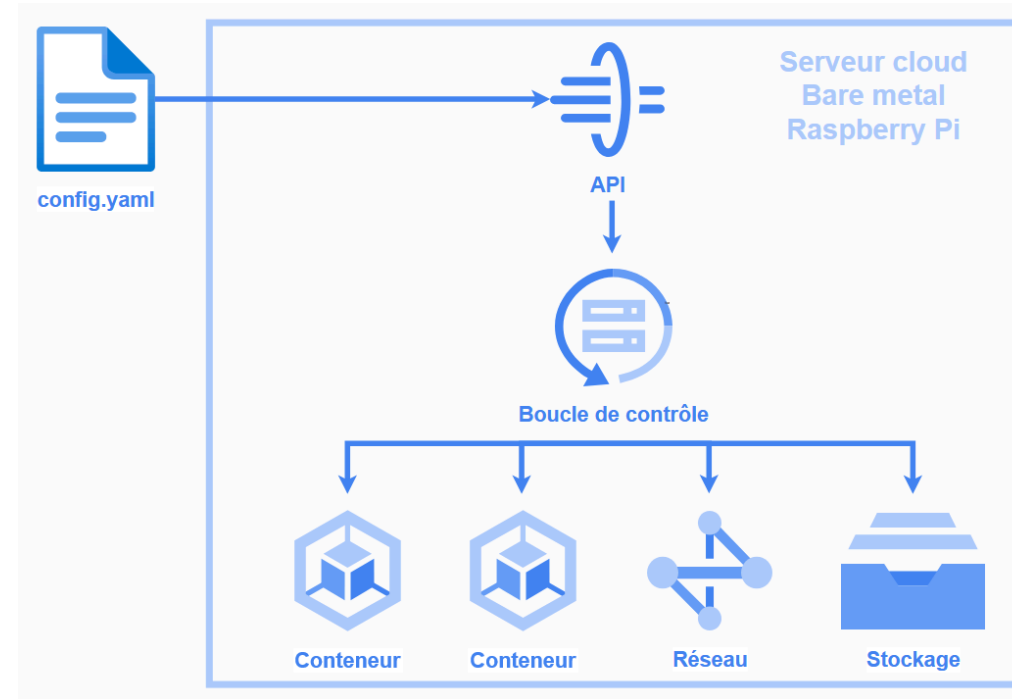
- Début: 02:50 Fin: **04:30**
- Cette configuration passe par un fichier unique: tout y est centralisé et défini, que ce soit les conteneurs, le réseau, le stockage, ou l'installation. >>>
- Le but est d'avoir un système entièrement déclaratif: vous déclarez simplement l'état dans lequel vous voulez que votre système soit, et il s'occupe lui-même d'y parvenir, sans que vous ayez à effectuer la moindre action. >>>

- Distribution Linux spécialisée
- Surface **minimale**: le strict nécessaire pour les conteneurs
 - Pas d'SSH, de shell, ou de commandes
 - Piloté par API
- Fichier de configuration unique
 - Système déclaratif
 - Homogène: bare-metal, VPS, embarqué, etc.

Introduction — Solution

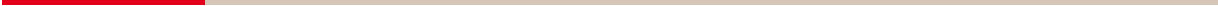
- Début: 02:50 Fin: **04:30**
- Enfin, un autre point très important est l'homogénéité: l'idée est que le même fichier de configuration puisse être utilisé sur plusieurs types d'environnements, que ce soit le cloud, le bare-metal, un Raspberry Pi, etc. >>>

- Distribution Linux spécialisée
- Surface **minimale**: le strict nécessaire pour les conteneurs
 - Pas d'SSH, de shell, ou de commandes
 - Piloté par API
- Fichier de configuration unique
 - Système déclaratif
 - Homogène: bare-metal, VPS, embarqué, etc.



- Début: 02:50 Fin: **04:30**
- Pour résumer grossièrement le fonctionnement:
 - vous écrivez un fichier de configuration
 - vous le transmettez à l'API
 - puis le système se charge de configurer les conteneurs, le réseau, le stockage, etc., selon ce que vous avez exprimé dans votre fichier.
- Pour mieux comprendre comment cela se passe, je vais vous en faire la démonstration. **III**

Démonstration



Démonstration

1. **Conception**
2. **Implémentation**
3. **Tests & Validation**
4. **Comparaison avec d'autres solutions**

— Suite de la présentation

- Début: 04:30 Fin: **04:50**
- Dans la suite de cette présentation, je vais d'abord vous présenter les concepts fondamentaux de cette solution, ainsi que quelques aspects clés de son implémentation.
- Ensuite, je parlerai de la façon dont la solution a été testée et validée, et de la manière dont ses performances ont été mesurées, avant de la comparer à deux autres solutions proches des besoins exprimés, à savoir Talos Linux et NixOS. **|||**

Conception

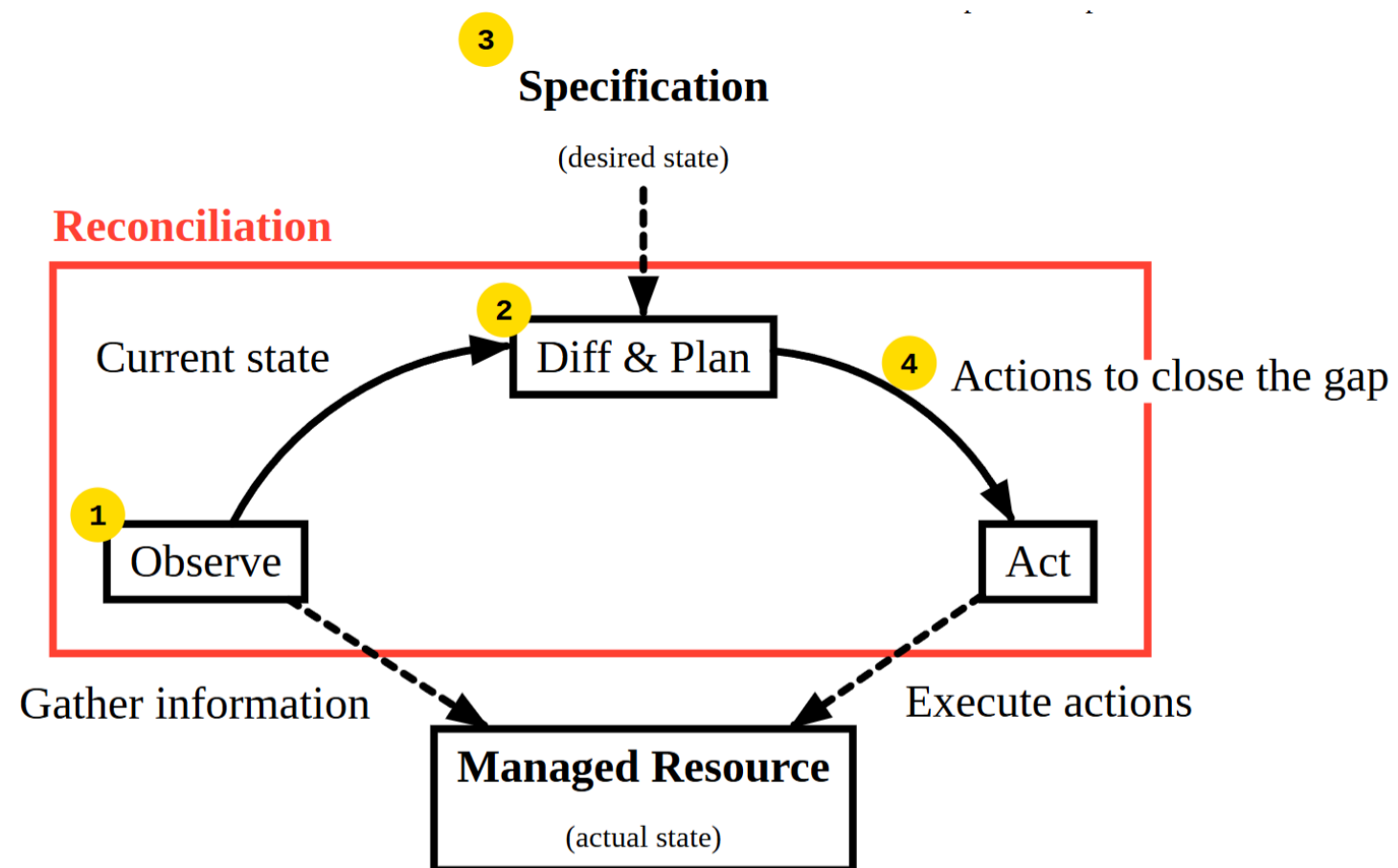
Conception

- Début: 04:50 Fin: **05:00**
- Et je vais donc maintenant vous présenter les concepts essentiels de cette solution, à commencer par la réconciliation. **|||**

- **Définition**: une boucle qui compare l'état actuel avec l'état désiré afin de les faire correspondre

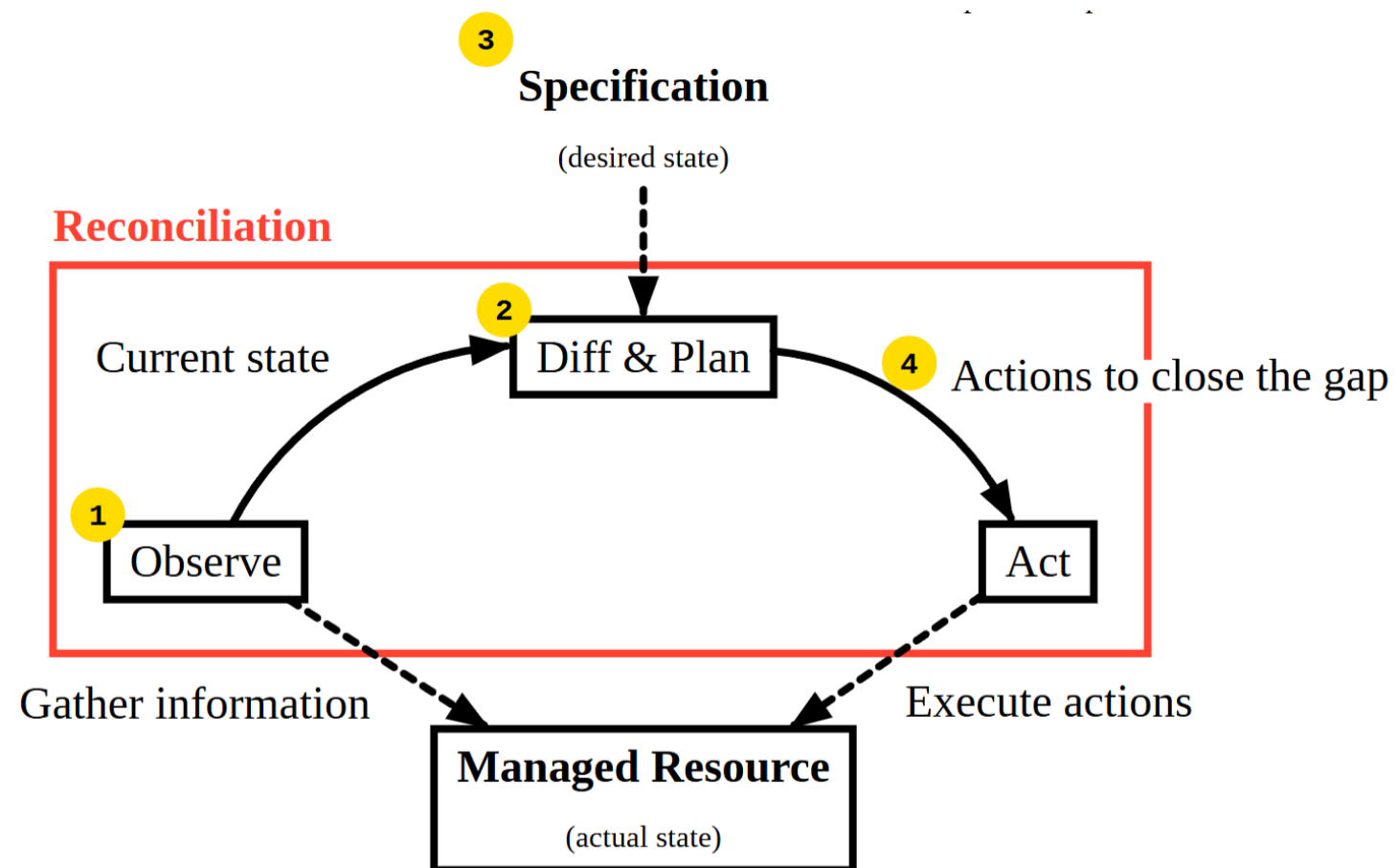
- Début: 05:00 Fin: **06:00**
 - En termes simples, la réconciliation est une boucle qui compare l'état actuel avec l'état désiré afin de les faire converger >>>
 - J'ai schématisé ici le fonctionnement général de cette boucle
1. D'abord, l'état actuel de la ressource à réconcilier est observé et capturé
 2. Cet état actuel est ensuite comparé avec l'état désiré, c'est-à-dire la configuration >>>

- **Définition**: une boucle qui compare l'état actuel avec l'état désiré afin de les faire correspondre



- Début: 05:00 Fin: **06:00**
 - En termes simples, la réconciliation est une boucle qui compare l'état actuel avec l'état désiré afin de les faire converger >>>
 - J'ai schématisé ici le fonctionnement général de cette boucle
1. D'abord, l'état actuel de la ressource à réconcilier est observé et capturé
 2. Cet état actuel est ensuite comparé avec l'état désiré, c'est-à-dire la configuration >>>

- **Définition**: une boucle qui compare l'état actuel avec l'état désiré afin de les faire correspondre



- Début: 05:00 Fin: **06:00**
- De cette comparaison découle un plan d'actions correctives à entreprendre pour faire converger les deux états
- S'il n'y a pas d'écart entre l'état actuel et l'état désiré, aucune action n'est nécessaire
- Ce plan d'action est ensuite exécuté sur la ressource gérée
- Cette exécution modifie l'état réel de la ressource, ce qui amène la boucle à observer de nouveau cet état
- Et comme ce cycle se répète à l'infini, tout écart ultérieur, qu'il soit dû à une panne ou à une modification externe, est automatiquement détecté et corrigé III

- Début: 06:00 Fin: **07:00**
- Je vais aussi brièvement aborder deux autres concepts essentiels que sont les ressources et les contrôleurs. >>>
- Une ressource est simplement un objet qui représente l'état désiré, et un instantané de l'état actuel
- Chaque ressource est identifiée par un schéma, et éventuellement un nom
- Le schéma permet de déterminer la structure de donnée de cet état désiré et de l'état actuel >>>

- **Ressource**: objet qui regroupe l'état désiré, et un instantané de l'état actuel

- Début: 06:00 Fin: **07:00**
- Je vais aussi brièvement aborder deux autres concepts essentiels que sont les ressources et les contrôleurs. >>>
- Une ressource est simplement un objet qui représente l'état désiré, et un instantané de l'état actuel
- Chaque ressource est identifiée par un schéma, et éventuellement un nom
- Le schéma permet de déterminer la structure de donnée de cet état désiré et de l'état actuel >>>

- **Ressource**: objet qui regroupe l'état désiré, et un instantané de l'état actuel
- **Contrôleur**: implémente la *réconciliation* pour une *ressource* donnée

- Début: 06:00 Fin: **07:00**
- Quant au contrôleur, il s'agit simplement de l'unité de code chargée d'implémenter la logique de réconciliation pour une ressource donnée.
- Dans le système, les contrôleurs, et par extension les ressources, sont regroupés en 3 domaines fonctionnels: >>>
- le domaine réseau, qui gère les routes, les adresses, le DHCP, etc. >>>
- le domaine conteneur qui gère, non seulement les conteneurs a proprement dit, mais aussi les réseaux de conteneurs, les images, etc. >>>

- **Ressource**: objet qui regroupe l'état désiré, et un instantané de l'état actuel
- **Contrôleur**: implémente la *réconciliation* pour une *ressource* donnée
 - Réseau

- Début: 06:00 Fin: **07:00**
- Quant au contrôleur, il s'agit simplement de l'unité de code chargée d'implémenter la logique de réconciliation pour une ressource donnée.
- Dans le système, les contrôleurs, et par extension les ressources, sont regroupés en 3 domaines fonctionnels: >>>
- le domaine réseau, qui gère les routes, les adresses, le DHCP, etc. >>>
- le domaine conteneur qui gère, non seulement les conteneurs a proprement dit, mais aussi les réseaux de conteneurs, les images, etc. >>>

- **Ressource**: objet qui regroupe l'état désiré, et un instantané de l'état actuel
- **Contrôleur**: implémente la *réconciliation* pour une *ressource* donnée
 - Réseau
 - Conteneur

- Début: 06:00 Fin: **07:00**
- Quant au contrôleur, il s'agit simplement de l'unité de code chargée d'implémenter la logique de réconciliation pour une ressource donnée.
- Dans le système, les contrôleurs, et par extension les ressources, sont regroupés en 3 domaines fonctionnels: >>>
- le domaine réseau, qui gère les routes, les adresses, le DHCP, etc. >>>
- le domaine conteneur qui gère, non seulement les conteneurs a proprement dit, mais aussi les réseaux de conteneurs, les images, etc. >>>

- **Ressource**: objet qui regroupe l'état désiré, et un instantané de l'état actuel
- **Contrôleur**: implémente la *réconciliation* pour une *ressource* donnée
 - Réseau
 - Conteneur
 - Système

- Début: 06:00 Fin: **07:00**
- et enfin le domaine système, qui gère les éléments qui n'ont pas été catégorisés dans les deux domaines **III**

- Début: 07:00 Fin: **08:00**
- Enfin, je vous ai dit tantôt que la réconciliation est un processus qui se répète à l'infini.
- Il faut donc décider qui est responsable de cette boucle. C'est ce que j'ai appelé l'orchestration.
- Et il y a deux modèles qui s'offrent à nous: le modèle centralisé et le modèle décentralisé >>>

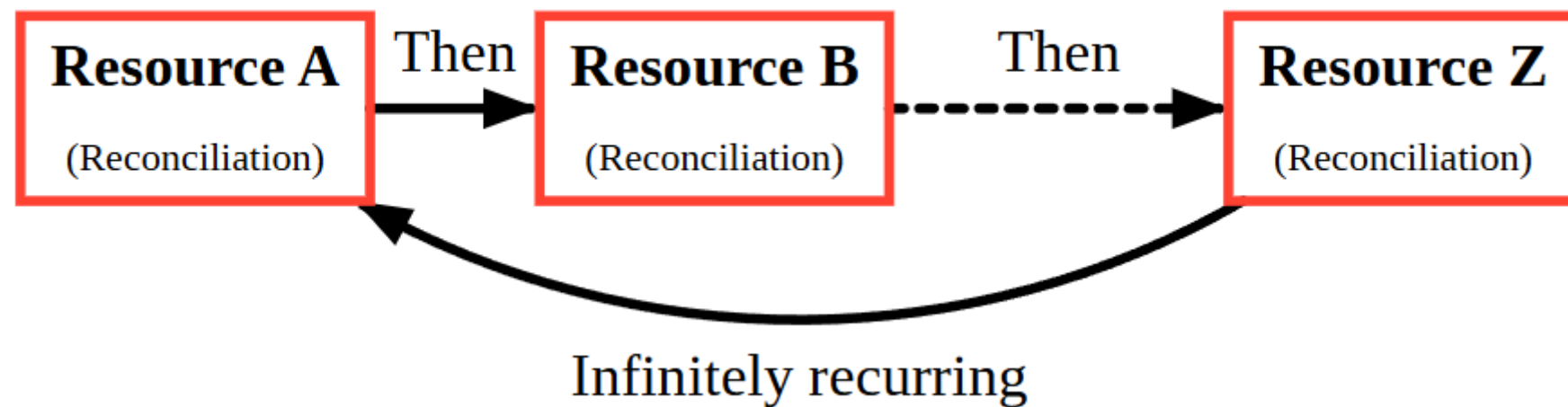
- Décentralisé: chaque contrôleur/ressource à sa propre boucle

- Début: 07:00 Fin: **08:00**
- Le modèle décentralisé est assez simple à expliquer:
- Chaque ressource étant gérée par un contrôleur, le contrôleur décide comme il souhaite de l'implémentation de la boucle.
- Par exemple, une seule boucle pour toutes ses ressources, une boucle par type, une boucle par ressource, etc. >>>
- Il a l'avantage d'être plus flexible, mais est plus compliqué à implémenter.
- C'est d'ailleurs le modèle d'orchestration adopté par Kubernetes >>>

- Décentralisé: chaque contrôleur/ressource à sa propre boucle
 - Plus flexible, mais plus compliqué à implémenter

- Début: 07:00 Fin: **08:00**
- Le modèle décentralisé est assez simple à expliquer:
- Chaque ressource étant gérée par un contrôleur, le contrôleur décide comme il souhaite de l'implémentation de la boucle.
- Par exemple, une seule boucle pour toutes ses ressources, une boucle par type, une boucle par ressource, etc. >>>
- Il a l'avantage d'être plus flexible, mais est plus compliqué à implémenter.
- C'est d'ailleurs le modèle d'orchestration adopté par Kubernetes >>>

- Décentralisé: chaque contrôleur/ressource à sa propre boucle
 - Plus flexible, mais plus compliqué à implémenter
- **Centralisé**: une boucle centrale qui contrôle tout
 - Plus rigide, mais plus simple à implémenter



- Début: 07:00 Fin: **08:00**
- Le modèle centralisé est l'exact opposé; dans ce modèle, un seul composant implémente la boucle, et va dispatcher la réconciliation vers le bon contrôleur.
- Et il possède donc les avantages et inconvénients inverses: >>>
- Il est plus rigide, mais plus simple à implémenter.
- Et c'est cette implémentation que j'ai retenue >>>

Implémentation

Implémentation

- Début: 07:30 Fin: **07:40**
- Maintenant que j'ai expliqué les concepts essentiels, je vais vous parler de l'implémentation
- En premier lieu, je vais vous donner une vue d'ensemble des technologies utilisées

- Basé sur aucune distribution/programme existant

- Début: 07:40 Fin: **08:40**
- Tout d'abord la solution ne se base sur aucune distribution ou programme existant: pas de Debian, Ubuntu, et pas non plus de systemd ou autre >>>
- Tout a donc été développé de zéro >>>
- Et pour ça j'ai choisi de le faire en Rust
- En ce qui concerne ce choix, il n'y a pas d'impératif particulier qui m'a forcé à prendre Rust; il fallait simplement un langage de bas niveau et Rust s'avère être celui avec lequel je suis le plus à l'aise >>>
- Il y a deux composants principaux que je n'ai pas redéveloppés >>>

- Basé sur aucune distribution/programme existant
 - Tout est développé de zéro

- Début: 07:40 Fin: **08:40**
- Tout d'abord la solution ne se base sur aucune distribution ou programme existant: pas de Debian, Ubuntu, et pas non plus de systemd ou autre >>>
- Tout a donc été développé de zéro >>>
- Et pour ça j'ai choisi de le faire en Rust
- En ce qui concerne ce choix, il n'y a pas d'impératif particulier qui m'a forcé à prendre Rust; il fallait simplement un langage de bas niveau et Rust s'avère être celui avec lequel je suis le plus à l'aise >>>
- Il y a deux composants principaux que je n'ai pas redéveloppés >>>

- Basé sur aucune distribution/programme existant
 - Tout est développé de zéro
- Langage de programmation: Rust

- Début: 07:40 Fin: **08:40**
- Tout d'abord la solution ne se base sur aucune distribution ou programme existant: pas de Debian, Ubuntu, et pas non plus de systemd ou autre >>>
- Tout a donc été développé de zéro >>>
- Et pour ça j'ai choisi de le faire en Rust
- En ce qui concerne ce choix, il n'y a pas d'impératif particulier qui m'a forcé à prendre Rust; il fallait simplement un langage de bas niveau et Rust s'avère être celui avec lequel je suis le plus à l'aise >>>
- Il y a deux composants principaux que je n'ai pas redéveloppés >>>

- Basé sur aucune distribution/programme existant
 - Tout est développé de zéro
- Langage de programmation: Rust
- Composants externes:

- Début: 07:40 Fin: **08:40**
- Tout d'abord, le runtime de conteneur, pour lequel j'ai choisi Podman
 - Ce choix a été fait durant le projet de semestre où je l'ai comparé à d'autres solutions, telles que containerd ou Docker, et Podman était la solution la plus simple à mettre en place dans le projet sans l'alourdir >>>
- Pour le bootloader, le composant qui charge l'OS, j'ai choisi Limine. Là encore, aucun impératif particulier. Je souhaitais simplement découvrir un autre outil que GRUB et celui-ci remplissait l'ensemble des besoins III

- Basé sur aucune distribution/programme existant
 - Tout est développé de zéro
- Langage de programmation: Rust
- Composants externes:
 - Runtime de conteneur: Podman

- Début: 07:40 Fin: **08:40**
- Tout d'abord, le runtime de conteneur, pour lequel j'ai choisi Podman
 - Ce choix a été fait durant le projet de semestre où je l'ai comparé à d'autres solutions, telles que containerd ou Docker, et Podman était la solution la plus simple à mettre en place dans le projet sans l'alourdir >>>
- Pour le bootloader, le composant qui charge l'OS, j'ai choisi Limine. Là encore, aucun impératif particulier. Je souhaitais simplement découvrir un autre outil que GRUB et celui-ci remplissait l'ensemble des besoins III

- Basé sur aucune distribution/programme existant
 - Tout est développé de zéro
- Langage de programmation: Rust
- Composants externes:
 - Runtime de conteneur: Podman
 - Bootloader: Limine

- Début: 07:40 Fin: **08:40**
- Tout d'abord, le runtime de conteneur, pour lequel j'ai choisi Podman
 - Ce choix a été fait durant le projet de semestre où je l'ai comparé à d'autres solutions, telles que containerd ou Docker, et Podman était la solution la plus simple à mettre en place dans le projet sans l'alourdir >>>
- Pour le bootloader, le composant qui charge l'OS, j'ai choisi Limine. Là encore, aucun impératif particulier. Je souhaitais simplement découvrir un autre outil que GRUB et celui-ci remplissait l'ensemble des besoins |||

- Racine immuable

- Début: 08:40 Fin: **10:40**
- Un autre point important de l'implémentation, c'est l'immuabilité du système de fichier.
- En effet, plus le système démarre dans un état connu, plus il est simple de l'administrer, et pour cela, j'ai choisi de rendre le système de fichier racine complètement immuable en utilisant SquashFs.
- Ainsi, à chaque redémarrage, on connaît exactement le contenu.
- Maintenant je vous l'accorde, ne rien pouvoir écrire sur le système de fichier n'est pas très pratique. >>>

- Racine immuable

- Début: 08:40 Fin: **10:40**
- Par exemple, si vous souhaitez changer la configuration DNS comme on l'a fait dans la démo, cela doit être écrit dans le répertoire /etc.
- Et on ne peut naturellement pas inclure ce répertoire dans l'image SquashFS, sinon on ne pourrait pas le changer. >>>
- Pour ce faire, j'ai superposé un système temporaire par-dessus l'image SquashFs en utilisant OverlayFS. >>>
- OverlayFS c'est sans doute quelque chose que vous avez déjà rencontré si vous utilisez des conteneurs, puisque c'est comme ça que la plupart fonctionnent >>>

- Racine immuable avec couche d'écriture temporaire

Implémentation — Immuabilité

- Début: 08:40 Fin: **10:40**
- Par exemple, si vous souhaitez changer la configuration DNS comme on l'a fait dans la démo, cela doit être écrit dans le répertoire /etc.
- Et on ne peut naturellement pas inclure ce répertoire dans l'image SquashFS, sinon on ne pourrait pas le changer. >>>
- Pour ce faire, j'ai superposé un système temporaire par-dessus l'image SquashFs en utilisant OverlayFS. >>>
- OverlayFS c'est sans doute quelque chose que vous avez déjà rencontré si vous utilisez des conteneurs, puisque c'est comme ça que la plupart fonctionnent >>>

- Racine immuable avec couche d'écriture temporaire
 - SquashFs, Tmpfs, OverlayFs

- Début: 08:40 Fin: **10:40**
- Par exemple, si vous souhaitez changer la configuration DNS comme on l'a fait dans la démo, cela doit être écrit dans le répertoire /etc.
- Et on ne peut naturellement pas inclure ce répertoire dans l'image SquashFS, sinon on ne pourrait pas le changer. >>>
- Pour ce faire, j'ai superposé un système temporaire par-dessus l'image SquashFs en utilisant OverlayFS. >>>
- OverlayFS c'est sans doute quelque chose que vous avez déjà rencontré si vous utilisez des conteneurs, puisque c'est comme ça que la plupart fonctionnent >>>

- Racine immuable avec couche d'écriture temporaire
 - SquashFs, Tmpfs, OverlayFs

- Début: 08:40 Fin: **10:40**
- Vous avez un ou plusieurs layer en lecture seule, ici notre système de fichier racine, puis un dernier layer en écriture.
- Vu que notre layer en écriture est temporaire, càd en RAM, a chaque redémarrage, on perd tout
- Très pratique, car on revient à un état connu, mais assez embêtant si vous devez tout reconfigurer a chaque fois. **>>>**
- Pour ce faire, deux répertoires sont montés sur des partitions ext4:
- /config qui contient la dernière config valide que vous avez envoyée
- et /var qui contient les données téléchargées, telles que les images

- Racine immuable avec couche d'écriture temporaire
 - SquashFs, Tmpfs, OverlayFs
- /config et /var persistés sur partitions ext4

- Début: 08:40 Fin: **10:40**
- Vous avez un ou plusieurs layer en lecture seule, ici notre système de fichier racine, puis un dernier layer en écriture.
- Vu que notre layer en écriture est temporaire, càd en RAM, a chaque redémarrage, on perd tout
- Très pratique, car on revient à un état connu, mais assez embêtant si vous devez tout reconfigurer a chaque fois. **>>>**
- Pour ce faire, deux répertoires sont montés sur des partitions ext4:
- /config qui contient la dernière config valide que vous avez envoyée
- et /var qui contient les données téléchargées, telles que les images

- Racine immuable avec couche d'écriture temporaire
 - SquashFs, Tmpfs, OverlayFs
- /config et /var persistés sur partitions ext4
- /etc et autres reconstruits à chaque redémarrage

Implémentation — Immuabilité

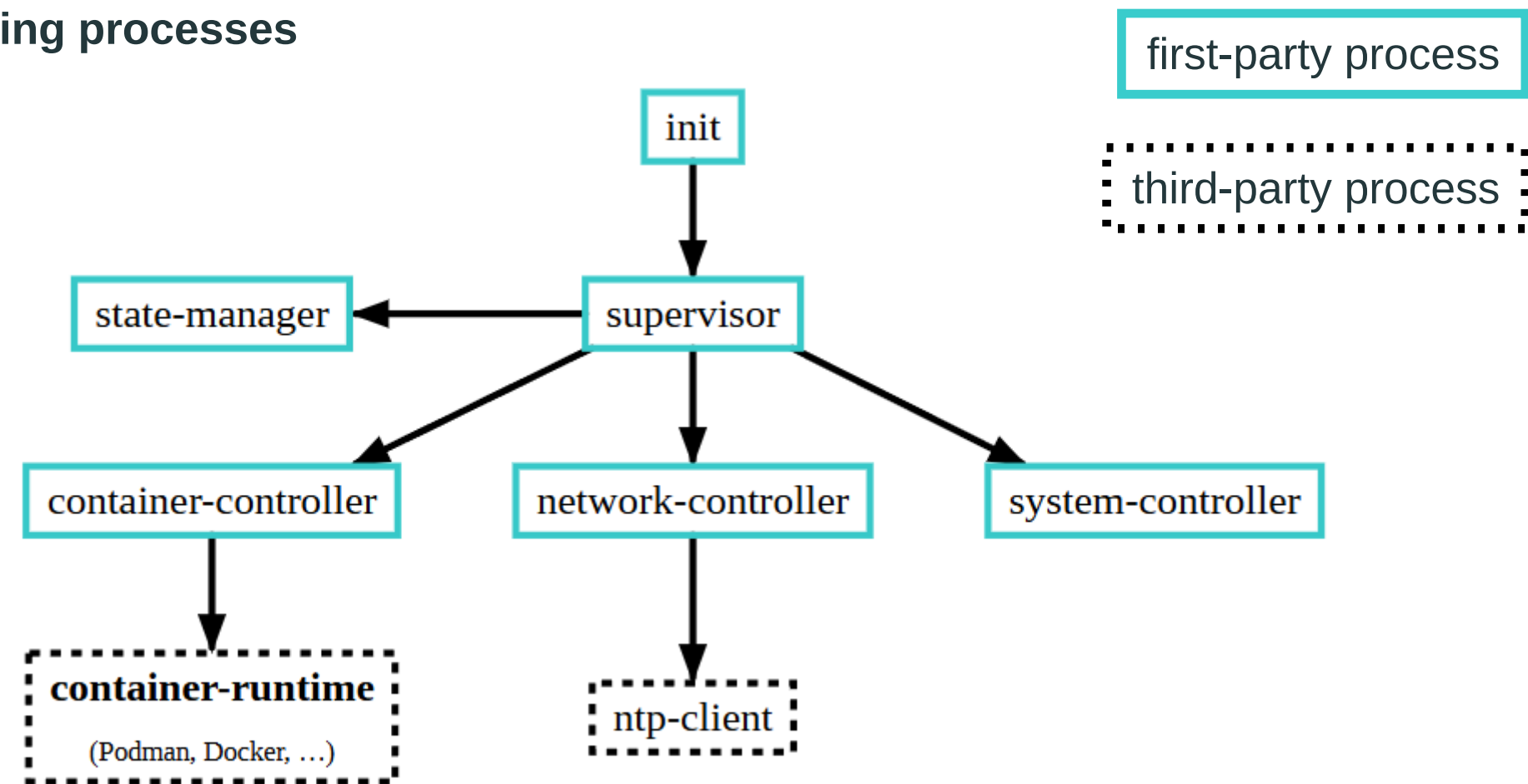
- Début: 08:40 Fin: **10:40**
- Tout le reste des dossiers et fichiers, comme /etc peut être reconstruit grâce a cela >>>
- A noter aussi que c'est a **vous** de choisir si vous souhaitez persister ces deux répertoires. Si vous les omettez de la config, vous obtenez un système complètement éphémère **III**

- Racine immuable avec couche d'écriture temporaire
 - SquashFs, Tmpfs, OverlayFs
- /config et /var persistés sur partitions ext4
- /etc et autres reconstruits à chaque redémarrage
- Persistance optionnelle: système peut être rendu entièrement éphémère

- Début: 08:40 Fin: **10:40**
- Tout le reste des dossiers et fichiers, comme /etc peut être reconstruit grâce a cela >>>
- A noter aussi que c'est a **vous** de choisir si vous souhaitez persister ces deux répertoires. Si vous les omettez de la config, vous obtenez un système complètement éphémère **III**

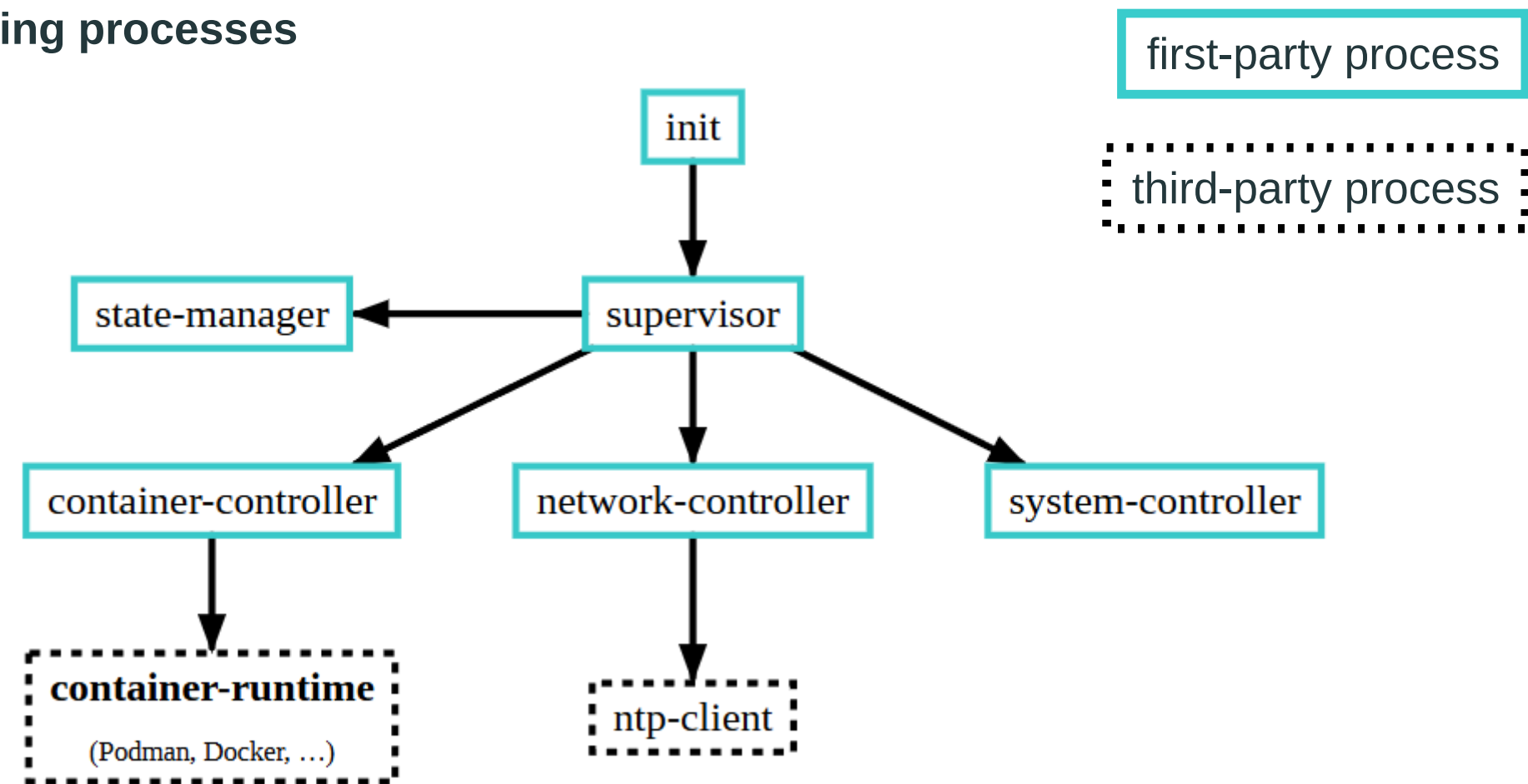
- Début: 10:40 Fin: **11:40**
- Enfin, en ce qui concerne la légèreté du système, cela est obtenu en minimisant le nombre de processus en cours d'exécution >>>
- Dans le mode de fonctionnement normal, càd hors installation, je vous montre ici l'ensemble des processus en cours d'exécution.
- Et j'insiste là-dessus, il n'y en a pas un seul de plus
- ça veut aussi dire que je peux rapidement vous faire le tour de ces composants.

All running processes



- Début: 10:40 Fin: **11:40**
- Enfin, en ce qui concerne la légèreté du système, cela est obtenu en minimisant le nombre de processus en cours d'exécution >>>
- Dans le mode de fonctionnement normal, càd hors installation, je vous montre ici l'ensemble des processus en cours d'exécution.
- Et j'insiste là-dessus, il n'y en a pas un seul de plus
- ça veut aussi dire que je peux rapidement vous faire le tour de ces composants.

All running processes

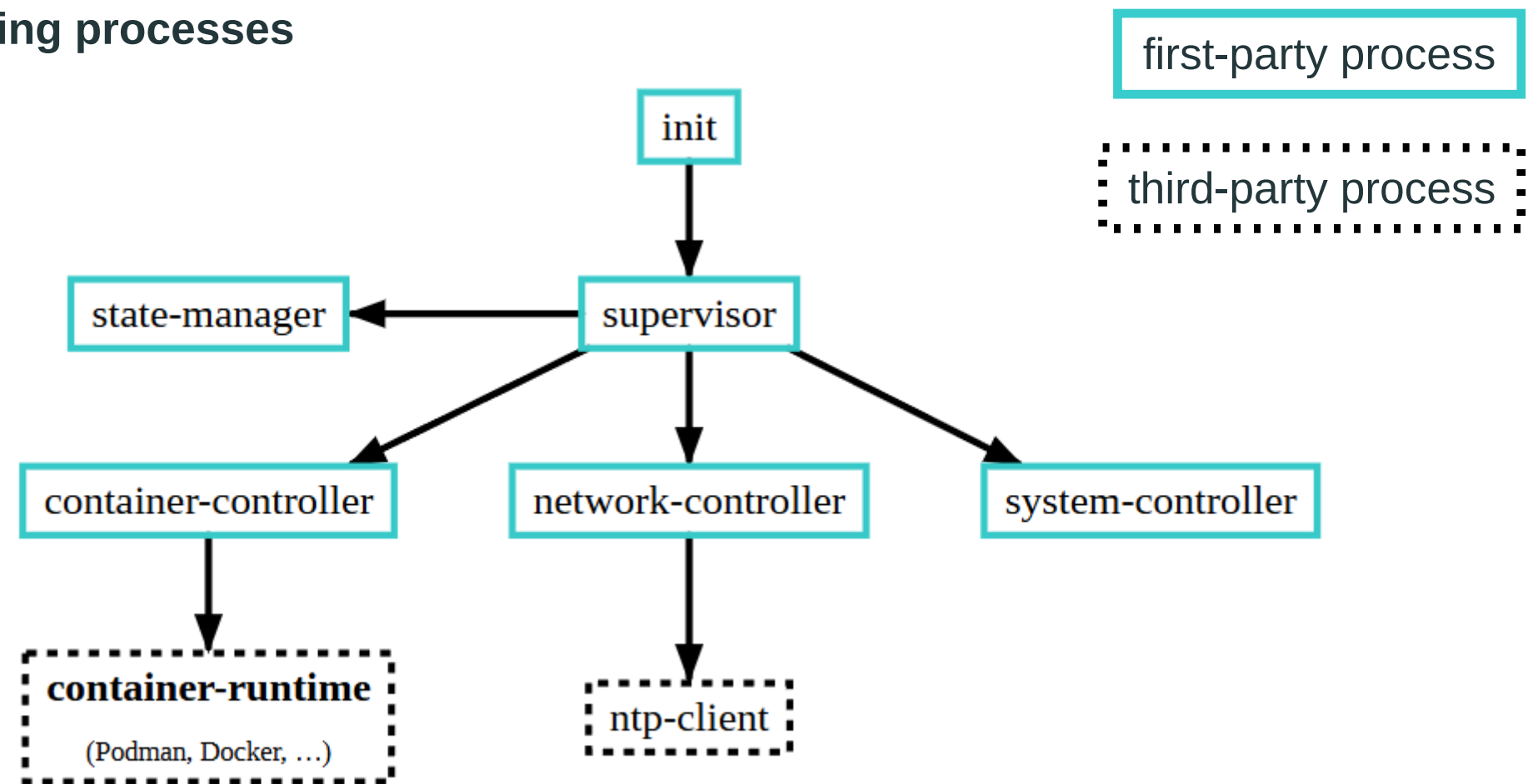


Implémentation

— Processus

- Début: 10:40 Fin: **11:40**
- En premier lieu vous avez l'init, qui est chargé de mettre en place ce fameux système de fichier immuable que je vous ai décrit juste avant
- Puis vous avez le superviseur. Son rôle est simplement de démarrer les composants principaux dans le bon ordre:
- C'est-à-dire démarrer tous les contrôleurs que je vous ai décrits précédemment
- Puis, une fois qu'ils sont tous prêts, démarrer le state-manager qui est le composant dans lequel la fameuse boucle d'orchestration et l'API sont implémentés.

All running processes



- Début: 10:40 Fin: **11:40**
- Et puis chaque contrôleur dispose de ses propres sous-processus, comme par exemple, le runtime de conteneur

- Compilation de multiples composants : noyau Linux, contrôleurs, init...

- Début: 11:40 Fin: **12:40**
- Enfin je vais vous parler du système de build.
- La particularité de ce projet c'est qu'il faut compiler non seulement une multitude d'applications (les contrôleurs, l'init, etc.) >>>
- Mais aussi le noyau Linux, puis packager tout ça sous différent format, comme des images SquashFS, eux-mêmes assemblé au final dans une image ISO >>>
- En outre, je vise plusieurs architectures: x64 et ARM >>>

- Compilation de multiples composants : noyau Linux, contrôleurs, init...
- Packaging en images SquashFS, assemblées en image ISO

- Début: 11:40 Fin: **12:40**
- Enfin je vais vous parler du système de build.
- La particularité de ce projet c'est qu'il faut compiler non seulement une multitude d'applications (les contrôleurs, l'init, etc.) >>>
- Mais aussi le noyau Linux, puis packager tout ça sous différent format, comme des images SquashFS, eux-mêmes assemblé au final dans une image ISO >>>
- En outre, je vise plusieurs architectures: x64 et ARM >>>

- Compilation de multiples composants : noyau Linux, contrôleurs, init...
- Packaging en images SquashFS, assemblées en image ISO
- Cible plusieurs architectures : x64 et ARM

- Début: 11:40 Fin: **12:40**
- Enfin je vais vous parler du système de build.
- La particularité de ce projet c'est qu'il faut compiler non seulement une multitude d'applications (les contrôleurs, l'init, etc.) >>>
- Mais aussi le noyau Linux, puis packager tout ça sous différent format, comme des images SquashFS, eux-mêmes assemblé au final dans une image ISO >>>
- En outre, je vise plusieurs architectures: x64 et ARM >>>

- Compilation de multiples composants : noyau Linux, contrôleurs, init...
- Packaging en images SquashFS, assemblées en image ISO
- Cible plusieurs architectures : x64 et ARM
- Système de build: Nix

Implémentation — Pipeline de build

- Début: 11:40 Fin: **12:40**
- Pour ce faire, j'ai choisi Nix.
- Je tiens ici à souligner que je parle de Nix en tant que système de build ou gestionnaire de paquet, mais pas de NixOS. >>>
- L'avantage de Nix c'est que cela donne des builds 100% reproductibles. C'est-à-dire que le build sur ma machine et le build sur un serveur de CI/CD donneront exactement le même résultat, à l'octet près.
- En outre, c'est assez simple de disposer du même environnement de dev >>>

- Compilation de multiples composants : noyau Linux, contrôleurs, init...
- Packaging en images SquashFS, assemblées en image ISO
- Cible plusieurs architectures : x64 et ARM
- Système de build: Nix
 - Builds 100% reproductibles

Implémentation — Pipeline de build

- Début: 11:40 Fin: **12:40**
- Pour ce faire, j'ai choisi Nix.
- Je tiens ici à souligner que je parle de Nix en tant que système de build ou gestionnaire de paquet, mais pas de NixOS. >>>
- L'avantage de Nix c'est que cela donne des builds 100% reproductibles. C'est-à-dire que le build sur ma machine et le build sur un serveur de CI/CD donneront exactement le même résultat, à l'octet près.
- En outre, c'est assez simple de disposer du même environnement de dev >>>

- Compilation de multiples composants : noyau Linux, contrôleurs, init...
- Packaging en images SquashFS, assemblées en image ISO
- Cible plusieurs architectures : x64 et ARM
- Système de build: Nix
 - Builds 100% reproductibles
- Inconvénient: recompilations longues avec Rust

- Début: 11:40 Fin: **12:40**
- L'inconvénient majeur, c'est la lourdeur de Nix. Bien qu'il y ait un système de cache, dans le cadre de Rust, l'unité qui est mise en cache est un programme entier. Donc, si vous modifier ne serait-ce qu'une ligne, tout va être recompilé. Combiner au Rust, ça peut faire des temps de build assez long. **|||**

Test & validation

Test & validation

- Début: 12:40 Fin: **12:50**
- Bon, les concepts et l'implémentation, c'est bien pratique, mais dans le cas d'espèce, si ça ne fonctionne pas correctement, on ne va pas aller très loin
- Dans cette section, je vais donc vous présenter brièvement comment la solution est testée et surtout quels sont les chiffres qu'on peut en tirer.

- 40 tests unitaires et intégrations

- Début: 12:50 Fin: **13:20**
- En ce qui concerne les tests unitaires et les tests d'intégration, c'est-à-dire lorsqu'un seul composant est testé, en isolation, il y en a une quarantaine >>>
- Ce qui est plus intéressant, c'est les 14 tests de bout en bout. >>>
- Ces tests exécutent une VM entière, avec un disque neuf, et toutes les interactions sont faites uniquement via l'API, de sorte à simuler un utilisateur réel |||

- 40 tests unitaires et intégrations
- 14 tests de bout en bout (E2E)

Test & validation — Stratégie de tests

- Début: 12:50 Fin: **13:20**
- En ce qui concerne les tests unitaires et les tests d'intégration, c'est-à-dire lorsqu'un seul composant est testé, en isolation, il y en a une quarantaine >>>
- Ce qui est plus intéressant, c'est les 14 tests de bout en bout. >>>
- Ces tests exécutent une VM entière, avec un disque neuf, et toutes les interactions sont faite uniquement via l'API, de sorte a simuler un utilisateur réel |||

- 40 tests unitaires et intégrations
- 14 tests de bout en bout (E2E)
 - VM isolée
 - Interaction uniquement via le client d'API

Test & validation — Stratégie de tests

- Début: 12:50 Fin: **13:20**
- En ce qui concerne les tests unitaires et les tests d'intégration, c'est-à-dire lorsqu'un seul composant est testé, en isolation, il y en a une quarantaine >>>
- Ce qui est plus intéressant, c'est les 14 tests de bout en bout. >>>
- Ces tests exécutent une VM entière, avec un disque neuf, et toutes les interactions sont faites uniquement via l'API, de sorte à simuler un utilisateur réel |||

- Sur 100 échantillons
- Même environnement que les tests E2E

- Début: 13:20 Fin: **14:20**
- En ce qui concerne les performances, cela a été mesuré sur 100 échantillons, et en suivant les mêmes protocoles que les tests E2E pour être au plus proche d'un usage réel, >>>
- Le système ne consomme que 160 mégas de RAM pour télécharger et exécuter un conteneur. Et si je ne mesure que l'OS en tant que tel, en omettant Podman, ce chiffre tombe en dessous des 80 mégas. >>>

- Sur 100 échantillons
- Même environnement que les tests E2E
- RAM: **160 MiB** pour un conteneur, **<80 MiB** pour le système seul
 - 160 MiB majoritairement dus à Podman

- Début: 13:20 Fin: **14:20**
- En ce qui concerne les performances, cela a été mesuré sur 100 échantillons, et en suivant les mêmes protocoles que les tests E2E pour être au plus proche d'un usage réel, >>>
- Le système ne consomme que 160 mégas de RAM pour télécharger et exécuter un conteneur. Et si je ne mesure que l'OS en tant que tel, en omettant Podman, ce chiffre tombe en dessous des 80 mégas. >>>

- Sur 100 échantillons
- Même environnement que les tests E2E
- RAM: **160 MiB** pour un conteneur, **<80 MiB** pour le système seul
 - 160 MiB majoritairement dus à Podman
- Rapidité:
 - **2.1s** cas idéal (machine moderne)
 - 19.3s installation (machine moderne)
 - 35s cas idéal (Raspberry Pi 1B [2008])

- Début: 13:20 Fin: **14:20**
- De même pour la vitesse, le démarrage du système, puis d'un conteneur « à chaud », et, par là, j'entends lorsque l'image est déjà téléchargée, ne prend que 2.1 secondes
- S'il est nécessaire de télécharger l'image, et bien cela prend au total 4.4s
- Et enfin, le cycle complet du démarrage de l'installer jusqu'à l'installation, le redémarrage, et le téléchargement du conteneur et son exécution prend 19.3s **III**

Comparaison avec d'autres solutions

Comparaison avec d'autres solutions

- Début: 14:20 Fin: **14:30**
- Enfin, je vais comparer ce qui a été développé a deux autres solutions: Talos Linux et NixOS >>>

— Solutions étudiées

- Début: 14:30 Fin: **15:30**
- Ces deux solutions sont les solutions identifiées comme les plus proches des besoins lors du projet de semestre. >>>
- La première, Talos Linux, est une distribution orientée Kubernetes: elle est déclarative et pilotée par API, mais son minimalisme reste « contraint » par les besoins de Kubernetes et vous le verrez, cela se répercute sur les chiffres. >>>

Talos Linux

- **Orienté Kubernetes**
- Déclaratif et piloté par API
- Minimaliste

Comparaison avec d'autres solutions — Solutions étudiées

- Début: 14:30 Fin: **15:30**
- Ces deux solutions sont les solutions identifiées comme les plus proches des besoins lors du projet de semestre. >>>
- La première, Talos Linux, est une distribution orientée Kubernetes: elle est déclarative et pilotée par API, mais son minimalisme reste « contraint » par les besoins de Kubernetes et vous le verrez, cela se répercute sur les chiffres. >>>

Talos Linux

- **Orienté Kubernetes**
- Déclaratif et piloté par API
- Minimaliste

NixOS

- **Générique**
- se base sur Nix
- Déclaratif, mais pas en continu
- Complexe à prendre en main

Comparaison avec d'autres solutions — Solutions étudiées

- Début: 14:30 Fin: **15:30**
- La seconde est NixOS. C'est une solution générique, se basant sur Nix, qu'on a déjà vu tout à l'heure.
- Et l'aspect particulier de NixOS c'est que vous « buildez » votre OS: quand vous faites un changement, c'est un nouveau build que vous appliquez à la machine
- Et c'est de là que vient l'aspect déclaratif, mais pas continu: il est possible de faire des changements hors du cadre du système de build
- Enfin, NixOS ce n'est pas la solution la plus facile à prendre en main

[illegible]

- Début: 15:30 Fin: **18:00**
- Ce tableau synthétise la comparaison sur six critères.
- Pour chaque solution, j'ai utilisé la configuration par défaut et fait les changements minimums pour pouvoir y déployer un conteneur.
- Aussi, ContainerOS correspond a la solution développée durant de travail, et les mesures de temps différent un petit peu de ce que je vous ai montré tout à l'heure, car cela a été fait sur un environnement différent.

Critères	ContainerOS	NixOS	Talos
Automatisation	✓	★	✓

- Début: 15:30 Fin: **18:00**
- Tout d'abord, l'automatisation, c'est le fait d'effectuer le moins d'actions possible sur l'ensemble du cycle de vie de la machine. Pour ça, ContainerOS et Talos, il ne suffit que de démarrer dans l'installeur et effectuer une seule commande, peu importe l'environnement
- Ce n'est toutefois pas le cas de NixOS, où l'installation initiale nécessite plusieurs actions manuelles >>>

Critères	ContainerOS	NixOS	Talos
Automatisation	✓	★	✓
Mémoire requise en exécution	160 MiB	276 MiB	1.4 GiB
Mémoire requise à l'installation	160 MiB	762 MiB	1.4 GiB

- Début: 15:30 Fin: **18:00**
- En ce qui concerne la mémoire, la limite acceptable est placée à 300 MiB.
- ContainerOS est nettement plus léger, aussi bien à l'exécution qu'à l'installation, avec 160 MiB durant les deux phases.
- Pour NixOS, l'exécution est légère, mais l'installation nécessitant de build certains aspects, cela consomme beaucoup de RAM
- Quand a Talos, le simple fait d'exécuter Kubernetes prend une quantité considérable de RAM.

Critères	ContainerOS	NixOS	Talos
Automatisation	✓	★	✓
Mémoire requise en exécution	160 MiB	276 MiB	1.4 GiB
Mémoire requise à l'installation	160 MiB	762 MiB	1.4 GiB
Rapidité d'installation	36s	300s	210s
Rapidité de démarrage	5.6s	31s	65s

- Début: 15:30 Fin: **18:00**
- Sur la rapidité, il n’y a pas de limites acceptables, mais vous pouvez voir que ContainerOS est clairement plus rapide que les autres solutions.
- Pour Talos Linux, c’est toujours lié à Kubernetes
- Et de même, pour NixOS, la lenteur à l’installation est toujours liée à cette notion de build. En revanche, la lenteur d’exécution est simplement due au fait que NixOS étant générique, la distribution démarre une multitude de services

Critères	ContainerOS	NixOS	Talos
Automatisation	✓	★	✓
Mémoire requise en exécution	160 MiB	276 MiB	1.4 GiB
Mémoire requise à l'installation	160 MiB	762 MiB	1.4 GiB
Rapidité d'installation	36s	300s	210s
Rapidité de démarrage	5.6s	31s	65s
Simplicité	✓	✓	✗

- Début: 15:30 Fin: **18:00**
- Enfin, sur la simplicité, ContainerOS et NixOS se rejoignent grâce à leur fichier de configuration unique, alors que Talos reste plus complexe du fait de son orientation cluster.
- Ce tableau confirme donc que pour le cas d'usage visé — un déploiement unitaire, non-cluster — ContainerOS répond de façon plus adéquate que les deux solutions existantes. **III**

Conclusion



Conclusion

—

- **But:** un OS pour déployer des conteneurs de manière simple

- **But:** un OS pour déployer des conteneurs de manière simple
- **Solution:** basé sur rien, piloté par API et 100% déclaratif

- **But:** un OS pour déployer des conteneurs de manière simple
- **Solution:** basé sur rien, piloté par API et 100% déclaratif
- **Résultats:**

- **But**: un OS pour déployer des conteneurs de manière simple
- **Solution**: basé sur rien, piloté par API et 100% déclaratif
- **Résultats**:
 - Rapide et très léger (160 MiB)

- **But:** un OS pour déployer des conteneurs de manière simple
- **Solution:** basé sur rien, piloté par API et 100% déclaratif
- **Résultats:**
 - Rapide et très léger (160 MiB)
 - Tous les objectifs ont été atteints

- **But:** un OS pour déployer des conteneurs de manière simple
- **Solution:** basé sur rien, piloté par API et 100% déclaratif
- **Résultats:**
 - Rapide et très léger (160 MiB)
 - Tous les objectifs ont été atteints
- **Difficultés:**
 - Bugs dans des composants externes (Podman, WSL, bibliothèque Rust)
 - Champ large
 - Évolution en terrain inconnu

- **Court terme:** mise à jour des composants + observabilité

- **Court terme:** mise à jour des composants + observabilité
- **Moyen terme:** plus de fonctionnalité dans les contrôleurs existants

- **Court terme**: mise à jour des composants + observabilité
- **Moyen terme**: plus de fonctionnalité dans les contrôleurs existants
- **Long terme**: système de plugin

- Très intéressant
 - Mise en pratique des technologies vues en cours
 - Domaine du développement jamais touché
- Projet personnel, amené à être maintenu dans le futur

Questions

Questions

h e p i a

Haute école du paysage, d'ingénierie
et d'architecture de Genève

Questions

—